

Highlights

Physics-aware Multitask Learning for Solving Parametric Partial Differential Equations

Jon Ander Rivera, Javier Del Ser

- We introduce a MTL architecture that uses multiple heads and a voter network to approximate parametric PDEs.
- We define two different training strategies Two-Phase and One-Phase.
- Our results reveal that Two-Phase training yields head specialization, while One-Phase training produces a more representative solution subspace.
- We obtain strong interpolation performance near the training range. However, both strategies exhibit limited extrapolation ability.

Physics-aware Multitask Learning for Solving Parametric Partial Differential Equations

Jon Ander Rivera^{a,*}, Javier Del Ser^{b,c}

^aDepartment of Applied Mathematics, University of the Basque Country (EHU), Bilbao, 48013, Spain

^bTECNALIA, Basque Research & Technology Alliance (BRTA), Derio, 48160, Spain

^cDepartment of Mathematics, University of the Basque Country (EHU), Leioa, 48940, Spain

ARTICLE INFO

Keywords:

Multitask Learning

Physics-Informed Neural Networks

Parametric Partial Differential Equations

ABSTRACT

Physics-Informed Neural Networks (PINNs) have emerged as a powerful framework for solving differential equations by embedding the governing physical constraints directly into neural network training. Their mesh-free formulation and compatibility with high-dimensional parameter spaces make them particularly well suited for parametric Partial Differential Equations (PDEs), where the objective is to learn families of solutions that vary smoothly across a range of parameter values. Recent work has shown that Multitask Learning provides a natural conceptual framework for such problems, as each parameter instance can be interpreted as a related task sharing common physical structure. In this work we investigate several multitask training strategies for PINNs applied to parametric PDEs. We provide an algebraic interpretation of the representations learned by the network, offering deeper insight into how PINNs encode parameter-dependent behavior. We further demonstrate that explicit specialization through task-specific heads is not required for accurate generalization across parameter values: when specialization is beneficial, it emerges implicitly during optimization, provided that the loss function is appropriately designed. These findings underscore the strong compatibility between multitask learning principles and parametric PINNs, pointing toward more interpretable, efficient, and generalizable approaches for scientific Machine Learning.

1. Introduction

In the last years, Physics-Informed Neural Networks (PINNs) have been widely shown to excel at solving Partial Differential Equations (PDEs) by embedding the physical constraints of the system directly into the loss function of a Neural Network (NN). In practice, PINNs include in the loss function the residuals of the governing PDE, as well as the corresponding initial and boundary conditions, so that the trained model yields solutions that inherently satisfy the underlying physics. This PINN-based formulation enables the unified treatment of direct problems (where the PDE is known and the goal is to compute the system response) and inverse problems (where unknown parameters or inputs of the PDE must be inferred from observed data). Their mesh-free nature, compatibility with automatic differentiation, and robustness in high-dimensional and/or data-scarce regimes have further contributed to their rapid development and adoption across a wide range of disciplines (Cai, Mao, Wang, Yin and Karniadakis, 2021a; Cai, Wang, Wang, Perdikaris and Karniadakis, 2021b; Raissi, Perdikaris and Karniadakis, 2019a).

Within the literature related to PINNs, several distinct research directions can be identified. A substantial body of work focuses on forward problems, where the governing PDE is known and the objective is to compute the corresponding system response. In parallel, other studies address inverse problems, which aim to infer unknown input parameters or physical coefficients from observed output data. Such inverse formulations are typically ill-posed, may admit non-unique solutions, and are highly sensitive to noise in the observations (Depina, Jain, Valssson and Gotovac, 2022; Baldan, Di Barba and Lowther, 2023; Difonzo, Lopez and Pellegrino, 2025). Beyond these two categories, several contributions consider the simultaneous treatment of forward and inverse problems within a unified PINN framework (Kapoor, Wang, Núñez and Dollevoet, 2024; Sahin, von Danwitz and Popp, 2024; Yu, Lu, Meng and Karniadakis, 2022). Another active research line explores hybrid approaches that combine PINNs with classical numerical methods for PDEs, such as the finite element method or Galerkin-based formulations, with the goal of improving accuracy, stability, and robustness (Kharazmi, Zhang and Karniadakis, 2019; Gao, Zahr and Wang, 2022;

*Corresponding author

 jonander.rivera@ehu.eus (J.A. Rivera); javier.delser@tecnalia.com (J. Del Ser)

ORCID(s): 0000-0002-0334-8454 (J.A. Rivera); 0000-0002-1260-9775 (J. Del Ser)

Liu, Zhang and Fuentes, 2026). Finally, growing attention has been devoted to uncertainty quantification in PINNs, seeking to characterize and control the uncertainty induced by model parameters, data scarcity, and approximation errors (Bharadwaja, Nabian, Sharma, Choudhry and Alankar, 2022; Yang, Meng and Karniadakis, 2021).

Building on the versatility of PINNs demonstrated for both forward and inverse problems, an important extension arises when the underlying PDE depends not only on spatial and/or temporal variables, but also on additional parameters. Parametric PDEs introduce one or more parameters that influence the behavior of the solution, for instance by modifying material properties, forcing terms, or geometric configurations (Cohen and DeVore, 2015). Solving such problems requires obtaining a solution that remains valid over a continuous range of parameter values, rather than at a single, fixed configuration. In practice, this is usually addressed by evaluating the PDE at multiple parameter instances within a prescribed range and constructing a surrogate model that captures how the solution varies across the entire parameter space.

PINNs naturally accommodate this setting: since parameters can be included as additional inputs to the neural network, a single trained model can learn families of solutions across the desired parametric domain. This capability makes PINNs particularly well suited for parametric PDEs, a fact corroborated by several recent studies. For example, Cao, Song and Zhang (2024) address a high-dimensional parametric engineering problem involving inviscid airfoil flow by solving a two-dimensional parametric version of the Euler equations, where the parameter encodes the flow conditions. In (Liu, Luo, Cheng, Liu, Li, Fan and Balachandar, 2024a), Parameterized Physics-Informed Neural Networks (P-PINNs) are used to infer flow fields, drag, and lift of spinning spheres over a multi-dimensional parameter space, demonstrating that training and deployment costs are substantially lower than running numerous conventional simulations across varying parameters. Further, Cho, Jo, Lim, Lee, Lee, Hong and Park (2024) introduce P²INNs, showing that they outperform standard PINNs in accuracy and parameter efficiency for 1D Convection–Diffusion–Reaction and 2D Helmholtz parametric problems. Complementary approaches include the work in (Penwarden, Zhe, Narayan and Kirby, 2023a), which leverages meta-learning and transfer learning techniques to accelerate the optimization process of parametric PINNs.

A key challenge when solving parametric PDEs with learning models lies in distinguishing between *interpolation* and *extrapolation* in the parameter domain. Ideally, the learned model should accurately interpolate between the parameter values used during training, producing physically consistent solutions for unseen parameters within the same range. However, generalizing beyond the training interval (i.e., extrapolation in the parameter space) is significantly more difficult and often unreliable, as it requires the model to infer solution behavior in regimes not directly constrained during training. This distinction is critical in applications such as design optimization, sensitivity analysis, and real-time control, where models are frequently queried at previously unseen parameter values. Consequently, understanding how learning-based solvers, and PINNs in particular, represent and generalize parametric dependence is central to assessing their robustness and practical applicability.

From a learning perspective, parametric PDEs naturally give rise to families of closely related solution instances, where each parameter configuration corresponds to a distinct but structurally similar problem. This observation establishes a direct connection with Multitask Learning (MTL), a Machine Learning paradigm in which a single model is trained to solve multiple related tasks simultaneously by exploiting shared structure and commonalities across tasks. From this perspective, PINNs themselves can be interpreted as a multi-task optimization framework: each component of the loss function (PDE residuals, initial and boundary conditions, observational data constraints) can be viewed as a distinct yet interdependent task. This conceptual alignment has motivated a growing body of research exploring how MTL strategies can enhance PINN training, improving convergence behavior, accuracy, and overall computational efficiency. Zhang and Hu (2025) introduce a multitask framework based on PINNs to address dynamic interconnected problems, designing a dual-channel parameter-sharing architecture capable of jointly solving forward and inverse problems. The study in (Bahmani and Sun, 2021) formulates PINN training as a multi-objective optimization task, combining a standard PINN loss with a secondary objective derived from a FEM-based reference solution. MTL techniques have also been applied to engineering applications. For instance, the work in (Zhao, Yao, Wang, Xie and Ablameyko, 2024) addresses fluid–structure interaction using a soft-parameter-sharing MTL architecture to better couple the underlying physical fields. Similarly, Wang, Qin, Shafiei, Dia, Mihaita and Grzybowska (2023) leverage auxiliary tasks in a multitask PINN to improve traffic-density prediction through knowledge transfer across tasks.

Importantly, the structure of parametric PDEs fits naturally within this MTL viewpoint. When each parameter configuration is interpreted as a separate but related task, MTL provides a natural mechanism to exploit shared physical behavior while capturing parameter-dependent variations. Thus, the intersection of MTL, PINNs, and parametric PDEs

represents a promising research direction: MTL can exploit correlations across parameterized tasks, while PINNs offer a physics-consistent mechanism for learning families of PDE solutions within a unified neural framework.

Building on this identified synergy, this work explores how MTL formulations can be leveraged to improve the training dynamics and interpretability of PINNs in parametric settings. In doing so, we introduce a MTL-PINN framework and analyze several training strategies that treat different parameter configurations as related tasks within a unified neural architecture. Our methodology enables an algebraic interpretation of what the network learns, providing additional insight into how the model organizes and shares information across the parameter space. A key finding is that, for parametric PDEs, it is often unnecessary to impose explicit specialization through task-specific heads: when such specialization is beneficial, it naturally emerges during optimization, provided that the loss function is appropriately designed. This observation reinforces the conceptual compatibility between MTL techniques and parametric PINNs, highlighting the potential of combining both to improve solution generalization, training stability, and interpretability. We pose the following research questions (RQ) to guide our study:

- RQ1: What do specialized and non-specialized heads actually learn from the domain spanned by the training parameter values? Is there a systematic relationship between the structure or shape of the PDE solutions and the representations learned by non-specialized heads?
- RQ2: To what extent does the relative performance of specialized versus non-specialized heads depend on the similarity (or distance) between the solutions associated with the training parameter values and those corresponding to unseen test parameters?

To answer these questions with empirical evidence, we perform extensive numerical experiments across several representative parametric PDEs, including Poisson’s equation, the Heat equation, and Burgers’ equation. Our results indicate that non-specialized heads tend to span a more expressive solution subspace, enabling a better representation of the variability observed across the family of parametric solutions. However, despite this advantage, neither explicitly specialized nor non-specialized formulations generalize reliably beyond the training parameter range, highlighting persistent limitations in extrapolation capability. Our findings suggest that, while MTL formulations can improve interpolation and representation efficiency in parametric PINNs, achieving robust extrapolation in parameter space remains an open challenge. This motivates further research toward new PINN–MTL formulations that can more effectively exploit shared physical structure, while enhancing generalization beyond the sampled parameter domain.

The remainder of the paper is organized as follows. Section 2 reviews the literature analyzing MTL, parametric PDEs and works that combine MT techniques, parametric PDEs, and PINNs. Section 3 states the problem that we will study. In Section 4 we define the MTL-PINN model architectures and training strategies considered in this paper. Sections 5 and 6 introduce the experiment design and show the numerical results, respectively. Finally, Section 7 covers the conclusions and future work.

2. Related Work

Before introducing the proposed MTL–PINN architecture, we briefly review the relevant literature surrounding three key concepts that underpin learning-based modeling of physical systems. Subsection 2.1 revisits MTL as a paradigm in which a single neural network learns multiple related tasks through shared representations, leading to improved generalization and parameter efficiency. Subsection 2.2 then introduces parametric PDEs, focusing on problems whose solutions depend on space, time, and tunable parameters, and discussing the computational challenges associated with high-dimensional parameter spaces. Subsection 2.3 reviews recent advances in physics-informed neural networks for parametric PDEs, including their use for surrogate modeling as well as extensions to inverse problems and uncertainty quantification. Finally, Subsection 2.4 describes recent attempts at solving parametric PDEs with PINN-based MTL approaches, setting the grounds for a closing statement about the contribution of our work.

2.1. Multitask Learning

MTL trains a single NN to perform multiple related tasks simultaneously. By sharing information across different tasks, it aims to improve efficiency and generalization compared to training separate models (Caruana, 1997). MTL leverages task relatedness to reduce overfitting and data requirements: shared layers capture common features, while task-specific layers model unique aspects of every task. This typically leads to improved performance through auxiliary supervision and greater parameter efficiency (Ruder, 2017).

Over the years, MTL has evolved from early NN approaches to modern Deep Learning (DL) architectures, incorporating techniques such as hard parameter sharing (common layers with task-specific heads), soft parameter sharing (regularized individual models), and gradient modulation strategies to balance task difficulties (Crawshaw, 2020). An in-depth survey classifies MTL algorithms into five main categories (Zhang and Yang, 2022): feature learning, low-rank approaches, task clustering, task relation learning, and decomposition methods. The survey also reviews their applications and theoretical foundations, highlighting the robustness of MTL across domains such as computer vision and scientific modeling.

In the context of modeling physical phenomena, MTL can learn surrogate models for multi-physics simulations, for example by predicting material microstructures under varying loads with physics-informed constraints (Wu, Bansal and Li, 2025), or by inverting seismic wave equations across multiple frequencies using shared convolutional encoders (Liu, Wang, Xi, Wang, Chen, Guo, Yan and Wang, 2024b).

2.2. Parametric Partial Differential Equations

Parametric PDEs describe physical systems whose solutions depend not only on spatial and/or temporal variables, but also on tunable parameters. These parameters may represent variations in material properties, forcing terms, source intensities, or boundary and initial conditions (Quarteroni, Manzoni and Negri, 2015). This is particularly useful in design optimization, real-time control, sensitivity analysis, and uncertainty quantification, where one seeks to rapidly evaluate how changes in parameters affect the solution. Common examples of parametric PDEs arise in heat diffusion (Rozza, Lassila and Manzoni, 2011), wave propagation (Glas, Patera and Urban, 2020), and fluid flow problems (Meng, Chen, Gao, Mu and Sun, 2025), where the parameters often correspond to thermal conductivity or diffusivity coefficients, wave speed or density distributions, viscosity values, inlet velocities, external loads, or pressure boundary conditions, among others.

Traditional numerical solvers such as the Finite Element Method (FEM), Finite Difference Method (FDM), or Finite Volume Method (FVM) discretize the spatial and temporal domains for each parameter instance, providing high accuracy. However, they become computationally expensive when exploring high-dimensional parameter spaces, suffering from the curse of dimensionality in high-dimensional parameter spaces due to their high computation costs (Brenner and Scott, 2008). Recent model order reduction strategies, such as reduced basis methods, mitigate these limitations by constructing low-dimensional approximations offline, enabling rapid evaluation for different parameter values during the online stage (Patera, Rozza et al., 2007).

2.3. Physics-Informed Neural Networks for parametric Partial Differential Equations

PINNs provide a flexible and mesh-free framework for solving parametric PDEs by approximating the unknown solution field with a NN. In this setting, the NN represents a continuous surrogate of the PDE solution and outputs the predicted state variable of interest (e.g., temperature, pressure, displacement, or velocity) at any given point in the spatio-temporal domain and for a given parameter configuration.

Instead of relying on labeled simulation data, PINNs incorporate the governing equations directly into the training process through a residual-based loss function. This residual is computed at collocation points by evaluating the error associated with the differential operator, the boundary conditions, and the initial conditions (Raissi, Perdikaris and Karniadakis, 2019b; Karniadakis, Kevrekidis, Lu, Perdikaris, Wang and Yang, 2021). In the parametric setting, the NN takes as input the spatial and temporal coordinates together with the problem parameters, and it is trained to minimize a loss function that penalizes violations of the PDE constraints across the parameter domain.

This formulation is particularly attractive for parametric PDEs because it enables learning a single surrogate model that generalizes across a continuous range of parameter values, avoiding the need to solve the PDE independently for each parameter instance. As a result, once trained, the model can be evaluated efficiently at inference time for new parameter configurations, making it well suited for design exploration, optimization, and real-time applications. Moreover, PINNs can naturally handle inverse problems by treating unknown physical coefficients as trainable variables. They can also address uncertainty quantification by incorporating uncertain parameters as additional inputs, and learning how parametric uncertainty propagates to the model outputs (Psaros, Meng, Zou, Guo and Karniadakis, 2023; Cuomo, Di Cola, Giampaolo, Rozza, Raissi and Piccialli, 2022).

2.4. PINN-based MTL for solving parametric PDEs

A few contributions have resorted to MTL to solve parametric PDEs using PINNs. The work in (Thanasutives, Numao and Fukui, 2021) proposes an enhanced framework for solving PDEs by integrating MTL and adversarial

training into PINNs. The approach leverages related auxiliary PDEs, generated through variations in equation parameters, to enable shared representation learning via a cross-stitch architecture, thereby improving generalization. To effectively balance multiple learning objectives, the authors employ uncertainty-weighted loss and gradient surgery techniques, which mitigate conflicts between task gradients and stabilize optimization. The work done in (Diao, Yang, Zhang, Zhang and Du, 2023) leverages MTL, parametric PDEs, and PINNs to address challenges in heterogeneous material simulations. MTL is applied through domain decomposition, where the computational domain is split into subdomains per material. Each one is modeled by a separate subnetwork. Parameter sharing across these subnetworks enhances accuracy by capturing shared physical patterns while allowing material-specific behaviors. Parametric PDEs arise implicitly from varying material properties across subdomains, enabling the framework to handle families of elasticity equations. PINNs form the core, embedding momentum balance, kinematic relations, and material-specific constitutive laws into the loss function of each subdomain network. The research presented by Penwarden, Zhe, Narayan and Kirby (2023b) applies MTL, parametric PDEs, and PINNs to accelerate solutions for families of PDEs. MTL emerges through meta-learning frameworks (a MTL technique). Parametric PDEs are central, defining tasks as variations in coefficients, boundary conditions, or source terms (e.g., in Poisson or Helmholtz equations), allowing the meta-trained model to approximate solutions over the entire parameter space efficiently without full retraining. PINNs provide the backbone, embedding the parameterized PDE residuals, boundary/initial conditions, and data losses into a unified objective.

Contribution. In this work, we develop a physics-aware MTL framework for solving parametric PDEs using PINNs. Specifically, we propose an architecture combining a shared backbone, multiple solution heads, and a voter network to represent families of parameter-dependent solutions as a learned basis. We introduce and compare two training strategies (One-Phase and Two-Phase) highlighting how explicit or implicit head specialization impacts the learned solution space. Through theoretical analysis and extensive experiments on benchmark PDEs, we demonstrate that explicit specialization is not necessary, as it can naturally emerge during optimization, and that jointly trained models yield richer and more expressive representations. Our results show strong interpolation capabilities across parameter ranges, while also identifying persistent challenges in extrapolation, thereby providing new insights into the role of multitask learning in parametric PINNs.

3. Problem Statement

Let $\mathcal{P} \subset \mathbb{R}^d$ be a parameter space of dimension d , and let $\Omega \subset \mathbb{R}^m$ be the spatial domain, with $t \in [0, T]$ denoting the time domain. A general formulation of a parametric PDE is defined as:

$$\mathcal{F}(u(\mathbf{x}, t; \boldsymbol{\mu}); \boldsymbol{\mu}) = f(\mathbf{x}, t; \boldsymbol{\mu}), \quad \forall \boldsymbol{\mu} \in \mathcal{P}, (\mathbf{x}, t) \in \Omega \times [0, T], \quad (1)$$

$$\mathcal{B}(u(\mathbf{x}, t; \boldsymbol{\mu}); \boldsymbol{\mu}) = g(\mathbf{x}, t; \boldsymbol{\mu}), \quad (\mathbf{x}, t) \in \partial\Omega \times [0, T], \text{ (boundary conditions, BC)} \quad (2)$$

$$u(\mathbf{x}, 0; \boldsymbol{\mu}) = u_0(\mathbf{x}; \boldsymbol{\mu}), \quad \mathbf{x} \in \Omega, \text{ (initial condition, IC)} \quad (3)$$

where $u : \Omega \times [0, T] \times \mathcal{P} \rightarrow \mathbb{R}^n$ denotes the parameter-dependent state (solution field), $\mathcal{F}(\cdot; \boldsymbol{\mu})$ is a (possibly nonlinear) differential operator, $\mathcal{B}(\cdot; \boldsymbol{\mu})$ encodes boundary conditions, and f , g , and u_0 represent parameter-dependent forcing, boundary data, and initial data, respectively. Therefore, the trained PINN acts as an implicit functional approximation $u_\theta(\mathbf{x}, t; \boldsymbol{\mu})$, mapping the spatial coordinates $\mathbf{x}$, time t , and parameters $\boldsymbol{\mu}$ to the corresponding PDE solution value $u(\mathbf{x}, t; \boldsymbol{\mu})$.

The parameter vector $\boldsymbol{\mu}$ may represent physical properties (e.g., material coefficients, forcing magnitudes, flow regime indicators) or other sources of variability. In some applications, $\boldsymbol{\mu}$ also affects the geometry. In that case, the PDE is posed on a parameter-dependent domain $\Omega(\boldsymbol{\mu})$ (and boundary $\partial\Omega(\boldsymbol{\mu})$). For simplicity of exposition we write (1)-(3) on a fixed reference domain Ω ; the extension to $\Omega(\boldsymbol{\mu})$ can be handled by introducing a mapping to a reference domain or by redefining the operators and data accordingly.

A PINN approximates the solution u by a neural surrogate represented as u_θ :

$$u_\theta(\mathbf{x}, t; \boldsymbol{\mu}) \approx u(\mathbf{x}, t; \boldsymbol{\mu}), \quad (4)$$

where θ denotes the NN parameters (*weights*). The PDE, boundary conditions, and initial condition are enforced through different residuals:

- PDE residual: Substituting u_θ into the differential operator in Expression (1), we get:

$$r_f(\mathbf{x}, t; \boldsymbol{\mu}) = \mathcal{F}(u_\theta(\mathbf{x}, t; \boldsymbol{\mu}); \boldsymbol{\mu}) - f(\mathbf{x}, t; \boldsymbol{\mu}). \quad (5)$$

- Boundary residual: Substituting u_θ in the boundary condition of Equation (2):

$$r_{bc}(\mathbf{x}, t; \boldsymbol{\mu}) = \mathcal{B}(u_\theta(\mathbf{x}, t; \boldsymbol{\mu}); \boldsymbol{\mu}) - g(\mathbf{x}, t; \boldsymbol{\mu}). \quad (6)$$

- Initial condition residual: proceeding analogously with u_θ in Equation (3), we obtain:

$$r_{ic}(\mathbf{x}; \boldsymbol{\mu}) = u_\theta(\mathbf{x}, 0; \boldsymbol{\mu}) - u_0(\mathbf{x}; \boldsymbol{\mu}). \quad (7)$$

Given collocation sets (which may depend on $\boldsymbol{\mu}$) $\{(\mathbf{x}_i, t_i)\}_{i=1}^{N_f} \subset \Omega \times [0, T]$, $\{(\mathbf{x}_j, t_j)\}_{j=1}^{N_{bc}} \subset \partial\Omega \times [0, T]$, and $\{\mathbf{x}_k\}_{k=1}^{N_{ic}} \subset \Omega$, the weights of the PINN solution to the PDE are learned (trained) by minimizing the sum of the squared residuals evaluated at such collocation points:

$$\mathcal{L}(\theta; \boldsymbol{\mu}) = \frac{1}{N_f} \sum_{i=1}^{N_f} |r_f(\mathbf{x}_i, t_i; \boldsymbol{\mu})|^2 + \frac{1}{N_{bc}} \sum_{j=1}^{N_{bc}} |r_{bc}(\mathbf{x}_j, t_j; \boldsymbol{\mu})|^2 + \frac{1}{N_{ic}} \sum_{k=1}^{N_{ic}} |r_{ic}(\mathbf{x}_k; \boldsymbol{\mu})|^2. \quad (8)$$

Therefore, the optimal NN weights to solve the PDE for a fixed parameter vector $\boldsymbol{\mu}$ are obtained by:

$$\theta^*(\boldsymbol{\mu}) = \arg \min_{\theta} \mathcal{L}(\theta; \boldsymbol{\mu}). \quad (9)$$

Above, we consider the parameter vector $\boldsymbol{\mu}$ of the residual of (8) to be a fixed value; it is not part of the training process. In a parametric PDE, the goal is not to approximate $u(\cdot, \cdot; \boldsymbol{\mu})$ at a single fixed parameter, but to learn a *family* of solutions over $\boldsymbol{\mu} \in \mathcal{P}$. To that end, we treat $\boldsymbol{\mu}$ as an additional input and train the network $u_\theta(\mathbf{x}, t; \boldsymbol{\mu})$ across a finite set of parameter samples:

$$\mathcal{P}_{\text{train}} = \{\boldsymbol{\mu}_\ell\}_{\ell=1}^{N_\mu} \subset \mathcal{P}, \quad (10)$$

by minimizing an empirical risk over parameters:

$$\theta^*(\mathcal{P}) = \arg \min_{\theta} \frac{1}{N_\mu} \sum_{\ell=1}^{N_\mu} \mathcal{L}(\theta; \boldsymbol{\mu}_\ell), \quad (11)$$

where the empirical risk is computed as in Expression (8), with $\boldsymbol{\mu}$ being an input variable to the neural surrogate throughout the computation of the residuals in (5)-(7). The loss in (11) enforces the governing equations over multiple parameter instances and encourages $u_\theta(\mathbf{x}, t; \boldsymbol{\mu})$ to capture the dependence of the solution on $\boldsymbol{\mu}$.

Once trained, the model is queried at unseen parameters $\boldsymbol{\mu} \notin \mathcal{P}_{\text{train}}$. When $\boldsymbol{\mu}$ lies *within* the region covered by the training samples (e.g., within their convex hull or within a prescribed training range), prediction can be interpreted as *interpolation* in parameter space. In contrast, when $\boldsymbol{\mu}$ lies *outside* that region, the model is required to *extrapolate*, which is typically more demanding because the governing constraints have not been enforced in that regime during training. In this work we use this distinction to assess how well the learned surrogate $u_\theta(\mathbf{x}, t; \boldsymbol{\mu})$ represents the parametric solution family and how reliably it generalizes beyond the training parameter domain.

4. Proposed MTL-PINN Model Architecture and Training Strategies

Building on the parametric PINN formulation introduced in Section 3, learning a surrogate model $u_\theta(\mathbf{x}, t; \boldsymbol{\mu})$ over a continuous parameter space $\mathcal{P}$ can be viewed as a threefold problem:

1. *Parameter sampling*: how to select a finite training set $\mathcal{P}_{\text{train}} \subset \mathcal{P}$ that is representative of the parameter domain, enabling accurate interpolation and, potentially, extrapolation;

2. *Shared solution learning*: given a fixed $\mathcal{P}_{\text{train}}$, how to construct a neural architecture $u_{\theta}(\mathbf{x}, t; \boldsymbol{\mu})$ capable of learning the family of PDE solutions associated with different parameter values while exploiting their common physical structure;
3. *Solution combination and interpretability*: how to combine and interpret the learned solutions for the different values of $\boldsymbol{\mu} \in \mathcal{P}_{\text{train}}$ in a way that provides insight into how parametric dependence is represented by the model.

In this work, we focus on the second and third aspects. The training parameter set $\mathcal{P}_{\text{train}}$ is assumed to be given, as is common in surrogate modeling and parametric PDE settings where parameter values are determined a priori. Our objective is therefore to study how a physics-informed multitask architecture can represent, share, and combine information across parameterized PDE instances.

From a MTL perspective, a parametric PDE naturally decomposes into a collection of related tasks. Each task corresponds to solving the same PDE for a fixed parameter value $\boldsymbol{\mu}$. Given a training set $\mathcal{P}_{\text{train}}$, we define the ℓ -th task as the problem of approximating the PDE solution $u(\mathbf{x}, t; \boldsymbol{\mu}_{\ell})$ associated with parameter value $\boldsymbol{\mu}_{\ell}$. The task loss is given by the PINN residual $\mathcal{L}(\theta; \boldsymbol{\mu}_{\ell})$ defined in Expression (8), where the parameters θ are given by the union of the *backbone* weights θ_{bb} (shared across all tasks) and the head-specific weights θ_h^{ℓ} . We further assume that all tasks share the same governing equations, boundary conditions, and physical constraints, differing only in the parameter $\boldsymbol{\mu}$. This induces strong task relatedness, making MTL a natural framework for parametric PINNs. However, we note that the overarching goal of solving a parametric PDE is not to solve these parameter-dependent tasks, but rather to construct a neural surrogate capable of generalizing (both interpolatively and extrapolatively) beyond the parameters in $\mathcal{P}_{\text{train}}$. Therefore, an additional step is needed to combine/fuse the solutions for each task and yield a solution for any query $(\mathbf{x}, t, \boldsymbol{\mu})$, with $\boldsymbol{\mu} \notin \mathcal{P}_{\text{train}}$.

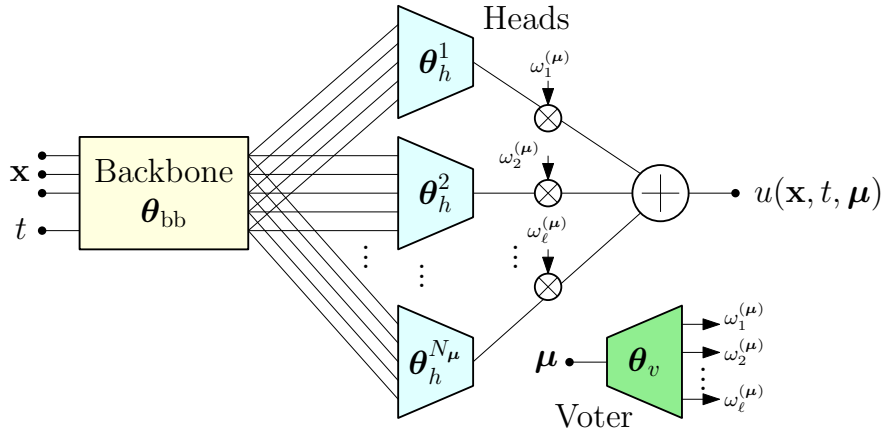


Figure 1: Architecture of the proposed MTL-PINN network designed to solve parametric PDEs.

Considering this rationale, we design the MTL-PINN architecture depicted in Figure 1, which is composed of three components: 1) a shared backbone network $f_{\theta_{\text{bb}}}(\cdot)$ with weights θ_{bb} ; 2) multiple output heads $f_{\theta_h^{\ell}}(\cdot)$ (one per training parameter, each with weights θ_h^{ℓ}), and 3) a linear combination of the head outputs via parameter-dependent weights $\omega_{\ell}^{(\mu)}$ estimated by a voter (or mixing) network $f_{\theta_v}(\cdot)$ with weights θ_v . The shared backbone extracts features common to all parameterized solutions. Each head produces a candidate solution field associated with one parameter $\boldsymbol{\mu}_{\ell} \in \mathcal{P}_{\text{train}}$, while the voter network outputs coefficients used to combine the head outputs. The final solution for a parameter $\boldsymbol{\mu}$ is given by:

$$u_{\theta}(\mathbf{x}, t, \boldsymbol{\mu}) = \sum_{\ell=1}^{N_{\mu}} \omega_{\ell}^{(\mu)} \cdot \left(f_{\theta_h^{\ell}} \circ f_{\theta_{\text{bb}}}(\mathbf{x}, t) \right), \quad \text{with } \{\omega_{\ell}^{(\mu)}, \dots, \omega_{N_{\mu}}^{(\mu)}\} = f_{\theta_v}(\boldsymbol{\mu}), \quad (12)$$

where $\circ$ denotes function composition. Voter weights are normalized via a softmax layer such that $\sum_{\ell=1}^{N_{\mu}} \omega_{\ell}^{(\mu)} = 1 \forall \boldsymbol{\mu}$. It is important to note that each head should not be necessarily regarded as being intrinsically tied to a specific parameter value $\boldsymbol{\mu}$. Instead, our formulation should be interpreted as a *learned basis*; under this view, the heads span

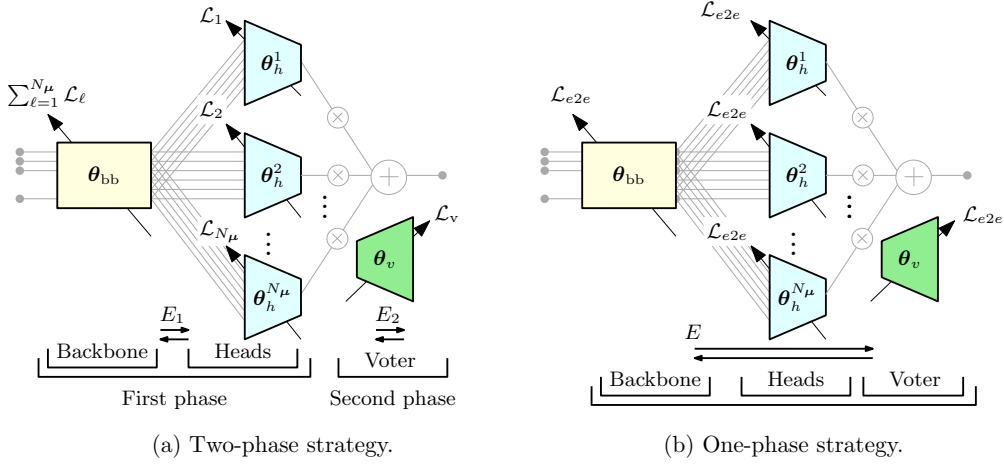


Figure 2: Training strategies for the proposed MTL-PINN architecture: (a) two-phase strategy; (b) one-phase strategy.

a low-dimensional subspace of the parametric solution manifold, and the voter network maps each parameter value μ to coordinates in this subspace. This enables both interpolation across the training parameters and an interpretable representation of how different solution components are activated as μ varies. Given this interpretation, we consider two training strategies corresponding to different inductive biases on the heads:

- **Two-phase training:** Under this strategy, head specialization is encouraged explicitly. Algorithm 1 and Figure 2.a describe this training strategy, which is structured in two consecutive phases, each designed to impose a specific inductive bias on the MTL-PINN architecture. The first phase (lines 3 to 19) promotes head specialization, whereas the second phase (lines 20 to 30) learns how to combine such specialized components for arbitrary parameter values through the voter:

- In the first phase, the voter network is frozen such that its parameters $\theta_v(\mu)$ remain fixed throughout optimization. Under this constraint, the training process focuses exclusively on learning the shared backbone θ_{bb} and the task-specific heads $\{\theta_h^\ell\}_{\ell=1}^{N_\mu}$. At each training epoch $e = 1, \dots, E_1$, collocation points for the PDE residual, boundary conditions, and initial conditions are sampled (line 6), and the loss $\mathcal{L}_\ell$ associated with each training parameter $\mu_\ell \in \mathcal{P}_{\text{train}}$ is computed independently, following the PINN formulation in Expression (8) (line 8). These losses are aggregated into a global backbone loss $\mathcal{L}_{bb} = \sum_{\ell=1}^{N_\mu} \mathcal{L}_\ell$ (line 9), which is used to update the shared backbone parameters via backpropagation once the heads have been frozen (lines 11 to 14). After updating the backbone, the training alternates over the heads (lines 15 to 18): for each ℓ , all head weights except θ_h^ℓ are frozen, and the corresponding loss $\mathcal{L}_\ell$ is backpropagated to update only the parameters of the ℓ -th head. This alternating optimization scheme encourages each head to specialize in approximating the solution associated with a specific training parameter, while the backbone captures features shared across all tasks.
- In the second phase, the backbone and all head parameters are frozen, and only the voter network is optimized. The voter parameters θ_v are trained using the loss defined in Expression (8) evaluated on the output of the voter given in Expression (12) (line 26). By accumulating this loss $\mathcal{L}_v^{bb-h-v}$ (backbone-head-voter) across all training parameter values (line 27), the resulting aggregate loss $\mathcal{L}_v$ evaluates the residuals of the combined solution produced by the voter across all training parameters, which is used to update its weights θ_v via backpropagation (line 29). Since the heads now form a fixed set of task-specialized functions, this phase learns a parameter-dependent combination of these components, allowing the model to interpolate across the parameter space by adapting the mixing coefficients produced by the voter network.

Overall, this two-phase procedure decouples representation learning and combination learning: the first phase constructs a set of specialized solution components aligned with individual training parameters, while the second phase learns how to linearly combine these components to approximate the parametric solution manifold. This explicit separation promotes interpretability, as each head can be associated with a distinct region of the parameter space, and the voter network provides insight into how these components are activated for unseen parameter values.

Algorithm 1: Two-phase training strategy: Head Specialization followed by voter training.

```

1 Input: Training parameters  $\mathcal{P}_{\text{train}} = \{\mu_1, \dots, \mu_{N_\mu}\}$ , MTL-based NN architecture composed by a shared
   backbone (parameters  $\theta_{\text{bb}}$ ), task-specific heads (parameters  $\theta_h^\ell$  for  $\ell \in \{1, \dots, N_\mu\}$ ), and voter weights  $\theta_v$ ;
   number of epochs  $E_1$  (Phase 1) and  $E_2$  (Phase 2); number of collocation points  $N_f$  (PDE residual),  $N_{\text{bc}}$ 
   (boundary residual),  $N_{\text{ic}}$  (initial condition residual)
2 Output: Trained MTL-PINN model  $u_\theta(\mathbf{x}, t, \mu)$ 

▷ Phase 1: Specialize heads (voter frozen)


3 Freeze voter network ( $\theta_v$  are not trainable)
4 for  $\text{epoch} = 1, \dots, E_1$  do
5     Set  $\mathcal{L}_{\text{bb}} = 0$ 
6     Sample  $\{(\mathbf{x}_i, t_i)\}_{i=1}^{N_f} \subset \Omega \times [0, T]$ ,  $\{(\mathbf{x}_j, t_j)\}_{j=1}^{N_{\text{bc}}} \subset \partial\Omega \times [0, T]$ , and  $\{\mathbf{x}_k\}_{k=1}^{N_{\text{ic}}} \subset \Omega$ 
7     for  $\ell = 1, \dots, N_\mu$  do
8         Compute the loss in Expr. (8) for parameter  $\mu_\ell$  through backbone  $f_{\theta_{\text{bb}}}(\cdot)$  and head  $f_{\theta_h^\ell}(\cdot)$ , yielding  $\mathcal{L}_\ell$ 
9         Accumulate the loss computed for  $\mu_\ell$ :  $\mathcal{L}_{\text{bb}} += \mathcal{L}_\ell$ 
10    end
11    Freeze all heads ( $\theta_h^\ell$  are not trainable  $\forall \ell \in \{1, \dots, N_\mu\}$ )
12    Unfreeze backbone ( $\theta_{\text{bb}}$  are trainable)
13    Backprop  $\mathcal{L}_{\text{bb}}$  to update  $\theta_{\text{bb}}$ 
14    Freeze backbone ( $\theta_{\text{bb}}$  are not trainable)
15    for  $\ell = 1, \dots, N_\mu$  do
16        Freeze all heads except head  $\ell$  ( $\theta_h^{\ell'}$  are not trainable  $\forall \ell' \neq \ell$ ,  $\theta_h^\ell$  is trainable)
17        Backprop  $\mathcal{L}_\ell$  to update  $\theta_h^\ell$  (parameters within head associated with training parameter  $\mu_\ell$ )
18    end
19 end

▷ Phase 2: Train voter network


20 Freeze backbone and all heads ( $\theta_{\text{bb}}$  and  $\theta_h^\ell \forall \ell \in \{1, \dots, N_\mu\}$  are not trainable)
21 Unfreeze voter ( $\theta_v$  are trainable)
22 for  $\text{epoch} = 1, \dots, E_2$  do
23     Set  $\mathcal{L}_v = 0$ 
24     Sample  $\{(\mathbf{x}_i, t_i)\}_{i=1}^{N_f} \subset \Omega \times [0, T]$ ,  $\{(\mathbf{x}_j, t_j)\}_{j=1}^{N_{\text{bc}}} \subset \partial\Omega \times [0, T]$ , and  $\{\mathbf{x}_k\}_{k=1}^{N_{\text{ic}}} \subset \Omega$ 
25     for  $\ell = 1, \dots, N_\mu$  do
26         Compute the loss in Expression (8) for parameter  $\mu_\ell$  through frozen backbone  $f_{\theta_{\text{bb}}}(\cdot)$ , frozen heads
            $f_{\theta_h^\ell}(\cdot)$  and trainable voter  $\theta_v$  as per Expression (12), yielding  $\mathcal{L}_{\text{bb-h-v}}^{\mu_\ell}$ 
27         Accumulate the end-to-end loss computed for  $\mu$ :  $\mathcal{L}_v += \mathcal{L}_{\text{bb-h-v}}^{\mu_\ell}$ 
28     end
29     Backprop  $\mathcal{L}_v$  to update  $\theta_v$ 
30 end
31 Return:  $u_\theta(\mathbf{x}, t, \mu)$  is given by the backbone, heads and voter NNs with weights  $\theta = \{\theta_{\text{bb}}, \{\theta_h^\ell\}_{\ell=1}^{N_\mu}, \theta_v\}$ 
    
```

- **One-phase training:** In contrast to the two-phase approach, the one-phase training strategy adopts a fully joint optimization scheme in which all components of the MTL-PINN architecture are trained simultaneously. This includes the shared backbone θ_{bb} , the set of task-specific heads $\{\theta_h^\ell\}_{\ell=1}^{N_\mu}$, and the voter network parameters θ_v .

According to Algorithm 2, at each training epoch, collocation points corresponding to the PDE residual, boundary conditions, and initial conditions are sampled (line 5). For each training parameter $\mu_\ell \in \mathcal{P}_{\text{train}}$, the model produces a solution by combining the outputs of all heads through the voter network, as defined in Expression (12). The resulting solution is used to compute a parameter-specific loss $\mathcal{L}_{\text{bb-h-v}}^{\mu_\ell}$ (line 7), following the PINN formulation in Expression (8), evaluated end-to-end across the entire architecture. These losses are aggregated (line 8) into a global *end-to-end* objective $\mathcal{L}_{\text{e2e}} = \sum_{\ell=1}^{N_\mu} \mathcal{L}_{\text{bb-h-v}}^{\mu_\ell}$, which incorporates contributions from all training parameters. A single backpropagation step is then applied to update all model parameters jointly (line 10).

From the perspective of representation learning, this one-phase training strategy does not enforce any explicit correspondence between heads and individual parameter values. Instead, the heads are optimized collectively as components of a learned basis spanning the solution manifold induced by the parametric PDE. The voter network then learns to map each parameter μ to a suitable combination of these basis elements. The joint optimization induces an implicit form of specialization, driven solely by the structure of the loss and the geometry of the parametric solution space. As a result, the learned heads tend to capture recurring patterns and modes of variation that are shared across multiple parameter configurations, rather than being restricted to individual tasks. This can lead to a more expressive and compact representation of the solution manifold, which can enhance interpolation performance across the parameter domain, albeit at the expense of reduced interpretability compared to the explicitly specialized formulation of the two-phase training strategy.

Algorithm 2: One-phase training strategy: joint MTL-PINN and voter training.

```

1 Input: Training parameters  $\mathcal{P}_{\text{train}} = \{\mu_1, \dots, \mu_{N_\mu}\}$ , MTL-based NN architecture composed by a shared
   backbone (parameters  $\theta_{\text{bb}}$ ), task-specific heads (parameters  $\theta_h^\ell$  for  $\ell \in \{1, \dots, N_\mu\}$ ), and voter weights  $\theta_v$ ;
   number of epochs  $E$ ; number of collocation points  $N_f$  (PDE residual),  $N_{\text{bc}}$  (boundary residual),  $N_{\text{ic}}$  (initial
   condition residual)
2 Output: Trained MTL-PINN model  $u_\theta(\mathbf{x}, t, \mu)$ 

▷ Single Phase: Train MTL-PINN and voter NNs


3 for  $\text{epoch} = 1, \dots, E$  do
4     Set  $\mathcal{L}_{\text{e2e}} = 0$ 
5     Sample  $\{(\mathbf{x}_i, t_i)\}_{i=1}^{N_f} \subset \Omega \times [0, T]$ ,  $\{(\mathbf{x}_j, t_j)\}_{j=1}^{N_{\text{bc}}} \subset \partial\Omega \times [0, T]$ , and  $\{\mathbf{x}_k\}_{k=1}^{N_{\text{ic}}} \subset \Omega$ 
6     for  $\ell = 1, \dots, N_\mu$  do
7         Compute the loss in Expr. (8) for parameter  $\mu_\ell$  through trainable backbone  $f_{\theta_{\text{bb}}}(\cdot)$ , trainable heads
            $f_{\theta_h^\ell}(\cdot)$  and trainable voter  $\theta_v$  as per Expression (12), yielding  $\mathcal{L}_\ell^{\text{bb-h-v}}$ 
8         Accumulate the loss computed for  $\mu_\ell$ :  $\mathcal{L}_{\text{e2e}} += \mathcal{L}_\ell^{\text{bb-h-v}}$ 
9     end
10    Backprop  $\mathcal{L}_{\text{e2e}}$  to update  $(\theta_{\text{bb}}, \{\theta_h^\ell\}_{\ell=1}^{N_\mu}, \theta_v)$ 
11 end
12 Return:  $u_\theta(\mathbf{x}, t, \mu)$  is given by the backbone, heads and voter NNs with weights  $\theta = \{\theta_{\text{bb}}, \{\theta_h^\ell\}_{\ell=1}^{N_\mu}, \theta_v\}$ 

```

Explicit head specialization as per the two-phase strategy offers interpretability and aligns naturally with classical MTL formulations. However, it may unnecessarily constrain the representation capacity of the model. When the solution manifold induced by a parametric PDE is low-dimensional but not aligned with individual parameter values, enforcing one head per parameter can be suboptimal. By contrast, learned basis representations obtained through one-phase training allow heads to capture shared solution patterns that recur across parameters. This perspective provides a unifying interpretation of our architecture and training strategies, and clarifies why non-specialized heads can yield stronger interpolation performance despite reduced task-level interpretability.

5. Experimental Setup

This section presents the experimental framework used to evaluate the proposed MTL-PINN architectures and training strategies. The design of our experiments is directly aligned with the research questions RQ1 and RQ2 posed in Section 1, and aims to provide systematic empirical evidence to support the claims and observations derived from our methodological analysis:

- To address RQ1, which concerns the nature of the representations learned by the model, we design experiments that expose the architecture to parametric PDEs exhibiting qualitatively different solution behaviors. Specifically, we consider parametric PDEs whose solutions range from smooth and regular to highly heterogeneous regimes characterized by sharp transitions or localized features. This controlled variation allows us to analyze how the different training strategies organize the learned solution components and to what extent they adapt to the intrinsic complexity of the underlying solution manifold.

- To tackle RQ2, which focuses on generalization with respect to the parameter domain, we evaluate the trained models across parameter values both inside and outside the range spanned by the training set. This distinction enables us to systematically compare interpolation and extrapolation behavior, thereby assessing the robustness of the learned representations when the model is queried in regions where the governing constraints have not been explicitly enforced during training.

The experiments are designed to isolate these two aspects while maintaining a consistent architectural configuration and training protocol across all scenarios. Each PDE is selected to emphasize specific characteristics relevant to the corresponding research question, and the parameter sampling strategies are defined accordingly to ensure a controlled and fair evaluation. In the following subsections, we present the mathematical formulation of the parametric PDEs considered in our study (Subsection 5.1, together with the specific NN architecture adopted for each problem and their training parameter settings (Subsection 5.2). Source code implementing the proposed MTL-PINN architecture, alongside scripts to run the experiments reported later in the manuscript, can be found at <http://github.com/...>

5.1. Parametric PDEs under consideration

The set of parametric PDEs used throughout the experimental evaluation has been selected to span a range of solution behaviors and structural properties, thereby providing a diverse suite to address RQ1 and RQ2. In particular, the chosen PDEs differ in terms of regularity, presence of sharp transitions, and sensitivity to parameter variations, allowing us to assess how the proposed architectures and training strategies adapt to diverse solution regimes. For each problem, we specify the governing equations, boundary and initial conditions, and the parameter configurations used for training:

- *Poisson's equation*: We first consider a parametric version of Poisson's equation, which serves as a canonical example of an elliptic PDE with smooth solution behavior. In this setting, the parameter μ controls the intensity or spatial structure of the forcing term, thereby inducing variations in the curvature and amplitude of the resulting solution. For each parameter value, the equation defines a steady-state field over the spatial domain, subject to prescribed boundary conditions. As the parameter varies, the corresponding solutions form a family of smooth functions with gradually changing profiles, making this problem particularly suitable for evaluating interpolation capabilities of the proposed MTL-PINN architecture. In particular, we solve the one-dimensional Poisson's equation given by:

$$-u_{xx}(x; \mu) + 2\mu = 0, \quad x \in [-1, 1], \quad (13a)$$

$$u(-1; \mu) = \mu, \quad (13b)$$

$$u'(1; \mu) = 2\mu, \quad (13c)$$

whose analytical solution is given by $u(x; \mu) = \mu x^2$. The spatial domain is defined as $\Omega = [-1, 1] \subset \mathbb{R}$, with no explicit temporal dependence. The problem is therefore purely spatial and one-dimensional ($m = 1$), and the solution takes as input the spatial coordinate $x \in \Omega$ together with the parameter μ . The values of μ used for training the proposed MTL-PINN network are $\mu = \{0.5, 1.5, 2.5, 3.5\}$, which are depicted in Figure 3.

From a learning perspective, the parametric Poisson equation provides a relatively well-behaved solution manifold, where variations across parameters are continuous and low-dimensional. This allows us to analyze how different training strategies capture shared structure across tasks and whether the learned heads specialize in distinct regions of the parameter space or instead represent common modes of variation.

- *Heat equation*: The second benchmark corresponds to a time-dependent parametric heat equation, modeling diffusion processes where the parameter governs the rate of heat propagation. In this case, the parameter μ modulates the diffusion coefficient, directly influencing how quickly the system evolves toward equilibrium. For each parameter configuration, the solution describes the temporal evolution of a temperature field, given an initial condition and boundary constraints. For simplicity we consider the one-dimensional Heat equation given by:

$$u_t(x, t; \mu) = \mu \cdot u_{xx}(x, t; \mu), \quad x \in [0, 1], t \in [0, 1] \quad (14a)$$

$$u(x, 0) = \sin(\pi x), \quad (14b)$$

$$u(0, t) = u(1, t) = 0, \quad (14c)$$

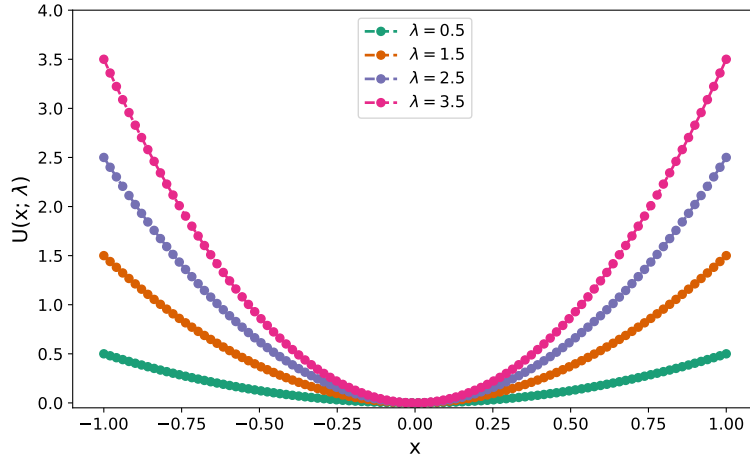


Figure 3: Analytical solutions of the parametric Poisson equation for the training parameters.

whose analytical solution is $u(x, t; \mu) = \sin(\pi x) \cdot e^{(-\pi^2 \mu t)}$. In this time-dependent problem, the spatial domain is defined as a one-dimensional interval $\Omega \subset \mathbb{R}$, and the temporal domain is given by $t \in [0, 1]$. These problems are thus defined over a space–time domain $\Omega \times [0, 1] \subset \mathbb{R}^2$, resulting in a two-dimensional input (x, t) augmented with the parameter μ . Consequently, the neural surrogate takes the form $u_\theta(x, t; \mu)$, and collocation points are sampled jointly over space and time. The values of μ used for training are $\{0.05, 0.1, 0.2, 0.3\}$. The solutions for such training parameters are plotted in Figure 4.

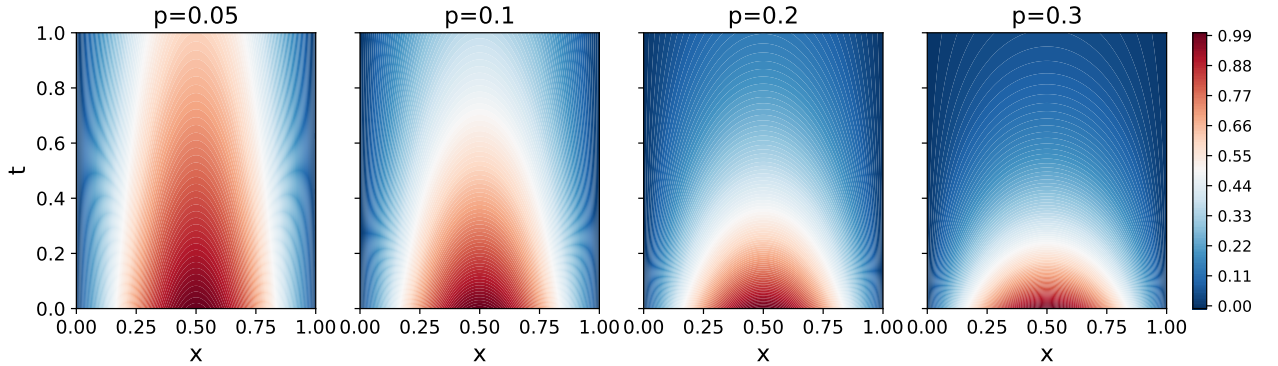


Figure 4: Analytical solutions of the parametric Heat equation for the training parameters.

- *Burgers' equation:* Finally, we consider a parametric version of Burgers' equation, which introduces nonlinear convection effects and leads to significantly more complex solution behaviors. The parameter controls the balance between diffusion and nonlinear advection, and as it varies, the solution transitions from smooth regimes to configurations characterized by sharp gradients and localized structures. This PDE is particularly relevant for evaluating the robustness of the proposed framework, as it produces heterogeneous solution regimes across the parameter space. For certain parameter values, the solution remains smooth and diffusion-dominated, while for others it develops steep fronts resembling shock-like transitions. As a result, the associated solution manifold is highly nonlinear and exhibits strong variations in local regularity. The one-dimensional Burgers' equation considered in our experiments is given by:

$$u_t + u \cdot u_x - \mu \cdot u_{xx} = 0, \quad x \in [-1, 1], t \in [0, 1], \quad (15a)$$

$$u(x, 0) = -\sin(\pi x), \quad (15b)$$

$$u(-1, t) = u(1, t) = 0, \quad (15c)$$

for which an analytical solution can be obtained through the Cole–Hopf transformation. As for the heat equation, this is a time-dependent problem with a one-dimensional spatial domain and a temporal domain $t \in [0, 1]$. The parameters used in the training are $\mu \in \{0.005, 0.01, 0.02, 0.03\}$. We represent the analytical solutions for the training parameter values in Figure 5.

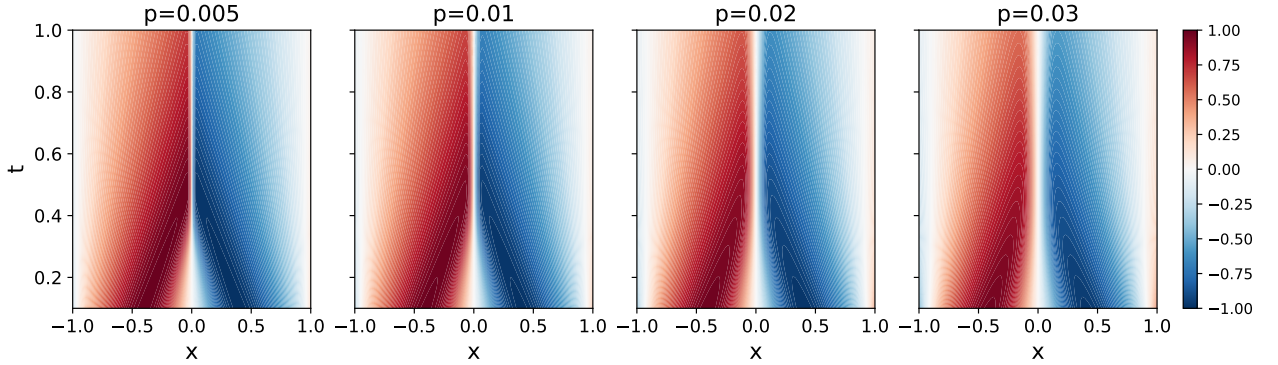


Figure 5: Analytical solutions of the parametric Burgers' equation for the training parameters.

The Burgers' equation permits us to analyze how the MTL-PINN architecture adapts to varying levels of solution complexity. It allows investigating whether head specialization emerges in response to distinct solution regimes, and to what extent joint training strategies are able to capture both smooth and highly localized behaviors within a unified representation.

5.2. Neural Architectures and Training Parameter Setting for the proposed MTL-PINN framework

The neural architectures and training configurations are adapted to the characteristics of each considered parametric PDE while maintaining a consistent design across experiments. As explained previously, the proposed MTL-PINN framework is composed of two main components: a *task completion module* (backbone + heads), responsible for approximating the solution for each training parameter; and a *solution interpolation module* (voter network), which estimates the coefficients to project the outputs of the different heads and predict the solution for unseen parameter values. In all cases the optimizer in use Adam with a learning rate of 0.001.

For Poisson's equation, the task completion module operates on a one-dimensional spatial domain, and therefore uses an input layer with one neuron. The backbone consists of 3 fully-connected layers with 64 neurons per layer, and $N_\lambda = 4$ heads are employed, corresponding to the number of training parameter values. The interpolation module takes as input the query parameter μ together with the task index ℓ , using an input layer of 2 neurons, followed by 2 fully-connected layers with 32 neurons per layer, and an output neuron representing the corresponding mixing weight $\omega_\ell^{(\mu)}$. For the case where we train heads and voter together (one-phase strategy, Algorithm 2), we perform $E = 30,000$ epochs. In the configuration where the voter network is trained separately (two-phase strategy, Algorithm 1), we use $E_1 = 30,000$ epochs for training the heads and then $E_2 = 5,000$ epochs for training the voter network. To evaluate the residual we employ $N_f = 100$ collocation points.

As for the heat and the Burgers' equation, the task completion section of the MTL-PINN framework is designed identically. The input layer has two neurons, as the problem solved is one-dimensional and time dependent. The backbone has 5 fully-connected layers with 128 neurons per layer. We have 4 heads (one per μ training value). On the other hand, solution interpolation part consists of an input layer with 2 neurons (query parameter μ and task index ℓ); 2 fully-connected layers with 64 neurons per layer; and an output layer with one neuron, producing $\omega_\ell^{(\mu)}$. For the heat equation, epochs are set to $E = 50,000$ (one-phase strategy), $E_1 = 50,000$ and $E_2 = 10,000$ (two-phase strategy). For the Burgers' equation, E , E_1 and E_2 are set to 100,000, 100,000, and 20,000, respectively. In both cases, to evaluate the residual we employ $N_f = 400$ collocation points for the general term, $N_{ic} = 100$ collocation points for the IC term, and another $N_{bc} = 100$ collocation points for the BC term. These points are randomly sampled at each epoch to mitigate overfitting (see lines 6 and 5 in Algorithms 1 and 2, respectively). Table 1 summarizes the configuration used for the experiments with every parametric PDE under consideration.

Table 1

Configuration for every parametric PDE included in our experiments.

Parametric PDE	MTL-PINN architecture										
	Task completion (backbone + heads)				Solution interpolation (voter)				Collocation points		
	Input dim.	Hid.lay. num.	Hid.lay. dim.	Head num.	Input dim.	Hid.lay. num.	Hid.lay. dim.	Output dim.	General term	BC	IC
Poisson	1	3	64	4	2	2	32	1	100	-	-
Heat	2	5	128	4	2	2	64	1	400	100	100
Burgers'	2	5	128	4	2	2	64	1	400	100	100

6. Numerical Results

We now present and discuss the results from our experiments informing our responses to RQ1 (Subsection 6.1) and RQ2 (Subsection 6.2).

6.1. RQ1: What do specialized and non-specialized heads actually learn?

We begin by analyzing the representations learned by the heads across the considered parametric PDEs, with the goal of understanding how different training strategies shape the internal structure of the solution space.

For problems with smooth solution manifolds, such as the Poisson and Heat equations, both training strategies yield heads whose outputs closely match the analytical solutions associated with different parameter values. This behavior can be observed in Figures X and Y, where each head produces smooth profiles that follow the expected curvature and amplitude variations induced by the parameter. In particular, heads trained under both strategies reproduce not only the global shape of the solution, but also the relative scaling across parameter values, indicating that the model correctly captures the parametric dependence within the training range. As a result, a clear form of specialization is observed in both training strategies, since individual heads can be visually associated with specific parameter regimes.

However, the nature of this specialization differs between strategies. In the Two-Phase approach, specialization is explicitly enforced by construction, as each head is optimized independently to approximate a specific parameter-dependent solution. This effect is clearly visible in Figures 6 and 7 (top rows), where each head aligns almost exactly with one of the ground-truth solutions, with minimal overlap across heads. In contrast, under the One-Phase strategy, no explicit assignment between heads and parameters is imposed. Nevertheless, as shown in Figures 6 and 7 (second row), the heads still exhibit a consistent form of specialization in these smooth regimes, producing solution profiles that closely resemble individual parameter instances. This indicates that, when the solution manifold is simple and low-dimensional, specialization can also emerge naturally under joint optimization.

The differences between both training strategies become more evident when considering more complex PDEs, such as the Burgers' equation. In this case, the solution manifold exhibits sharp gradients and strong nonlinearities, and the behavior of the heads differs significantly across training strategies. Under the Two-Phase regime, heads remain clearly specialized, each reproducing the solution associated with its corresponding training parameter, including the location and intensity of steep fronts (Figure 8, top row). These fronts shift consistently with the parameter, and each head captures one such configuration in isolation.

By contrast, in the One-Phase regime, the notion of specialization weakens noticeably. As illustrated in Figure 8 (second row), the outputs of the heads no longer align with individual parameter solutions. Instead, they display partial structures—such as smoothed fronts, localized gradients, or intermediate transition patterns—that cannot be directly associated with any single ground-truth solution. These outputs often lie between different solution regimes, effectively interpolating structural elements across parameters. This indicates that, in complex nonlinear settings, the heads behave as components of a shared basis rather than as task-specific approximators.

To further characterize these differences, we analyze whether the heads span a subspace capable of representing the variability of the parametric solution family. To this end, we compute the covariance matrix of the analytical solutions across parameter values, and compare it with the subspace spanned by the outputs of the learned heads. The alignment between both is quantified via the explained variance ratio, which measures the fraction of total variability captured by the head subspace. The evolution of this metric across different space and time discretization levels is reported in Table 2, where higher values of the explained variance ratio indicate a better match between the learned representation

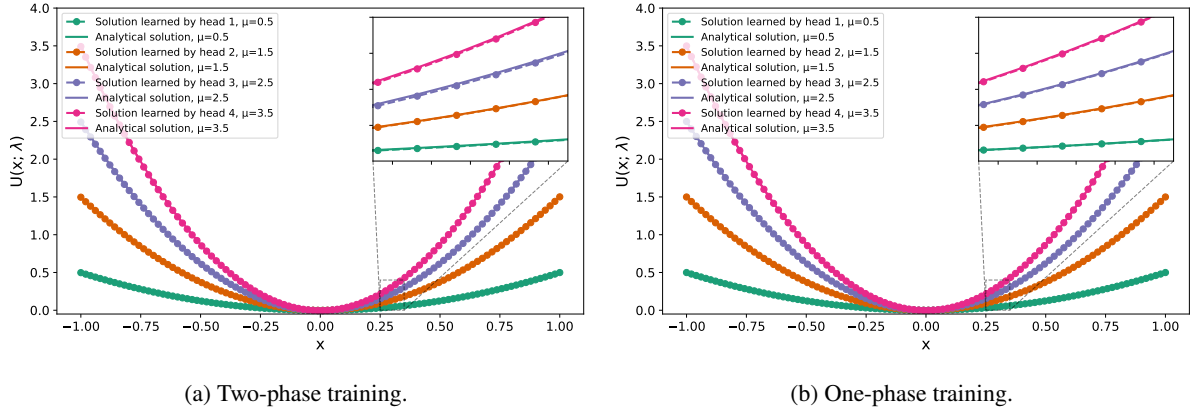


Figure 6: Heads learned by MTL-PINN for the Poisson problem under the two different training strategies described in Section 4.

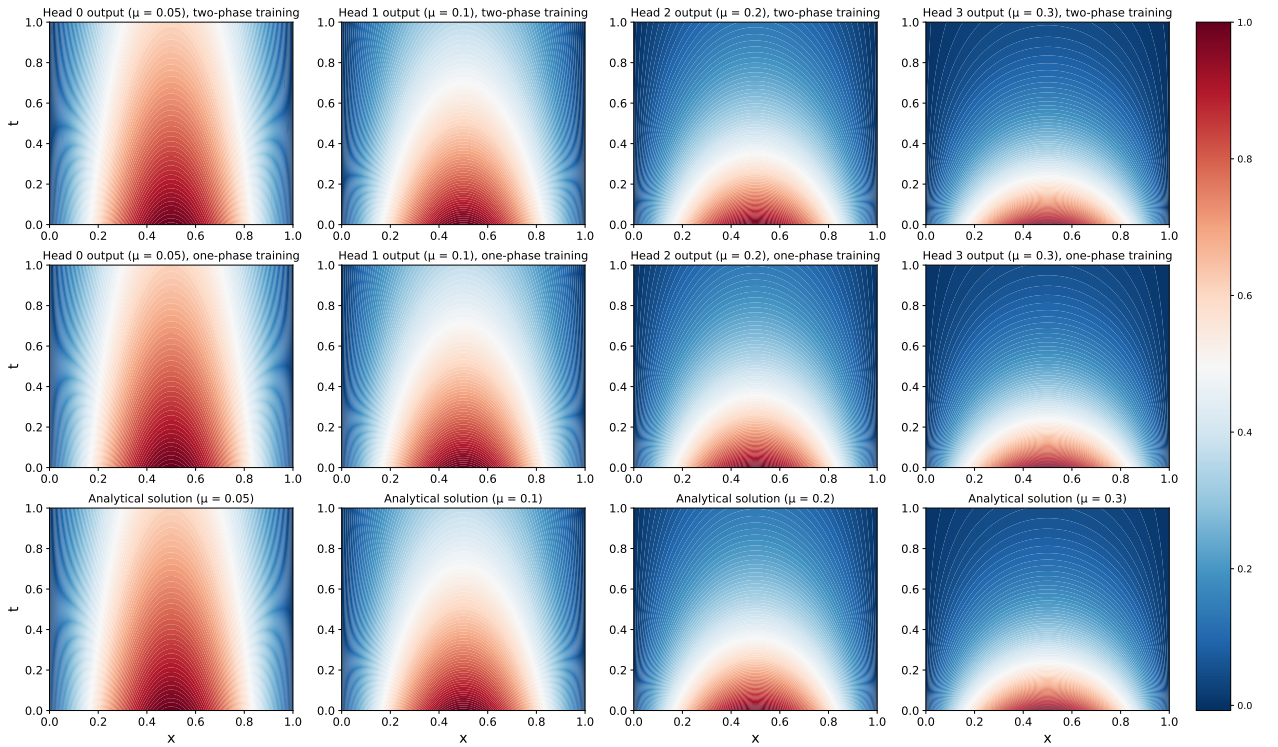


Figure 7: Heads learned by MTL-PINN for the heat problem under the two different training strategies described in Section 4: first row, two-phase training; second row, one-phase training; third row: analytical solution.

and the intrinsic structure of the solution manifold. This indicates that, although less interpretable in complex regimes, the non-specialized heads span a richer and more expressive subspace that better captures the intrinsic variability of the solution manifold. In contrast, enforcing one-to-one correspondence between heads and parameter values leads to a basis that is less aligned with the underlying geometry of the solution family, as reflected by the lower explained variance values.

On a concluding note on RQ1, our findings suggest that the notion of “specialization” depends both on the training strategy and on the complexity of the underlying parametric PDE. In simple, smooth regimes, both strategies yield clearly specialized heads, even when specialization is not explicitly enforced. However, as the solution manifold becomes more complex, only the Two-Phase strategy preserves explicit task-level specialization, while the One-Phase

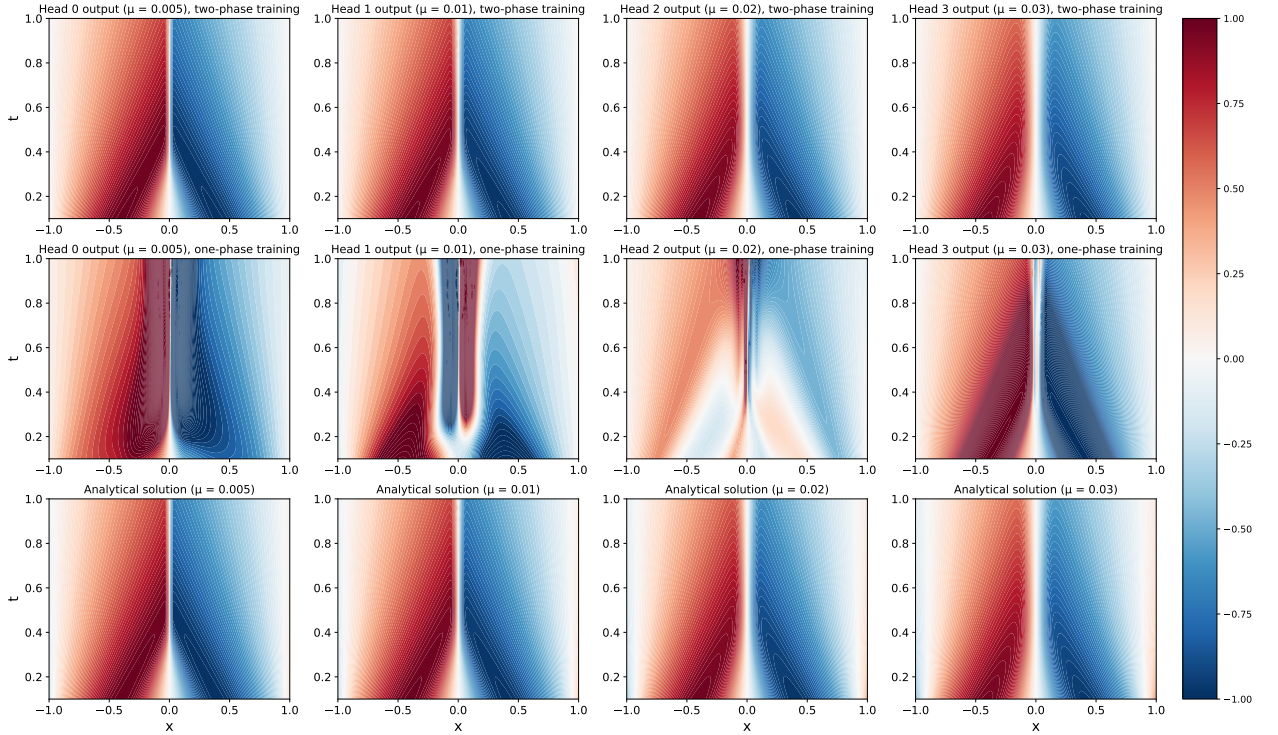


Figure 8: Heads learned by MTL-PINN for the Burgers' problem under the two different training strategies described in Section 4: first row, two-phase training; second row, one-phase training; third row: analytical solution.

Table 2

Comparison of the explained-variance metric between the specialized-heads model (Two-Phase strategy) and the non-specialized-heads model (One-Phase strategy) across three spatial discretizations and the Burgers' parametric PDE.

Mesh discretization		Explained Variance Ratio	
Space x	Time t	Two-Phase strategy	One-Phase strategy
30	30	0.725	0.792
50	50	0.749	0.809
100	100	0.754	0.819

approach transitions toward a basis-like representation, where heads encode shared modes of variation across the parameter space. Hence, for parametric PDEs, learning a suitable basis for the solution manifold is ultimately more relevant than assigning each head to a specific parameter value.

6.2. RQ2: How does MTL-PINN perform when predicting the solution of unseen query parameters?

We now analyze the generalization capabilities of the proposed MTL-PINN architectures with respect to the parameter domain, addressing RQ2. In particular, we evaluate how the learned models behave when queried at parameter values that lie both within (*interpolation*) and outside (*extrapolation*) the range spanned by the training set. This analysis is conducted separately for each PDE, with the goal of identifying consistent patterns across problems with different levels of complexity.

We begin with the Poisson equation, whose results are reported in Figure 9 and Table 3. For this problem, which is characterized by a smooth and low-dimensional solution manifold, both training strategies exhibit strong interpolation performance (results for $\mu = 2$). The predicted solutions closely match the analytical references for parameter values within the training range, accurately capturing both the shape and scaling of the solution. However, when evaluating *extrapolation* ($\mu = 8$), deviations from the analytical solution become visible, particularly as the distance from the

Table 3

Weights assigned by the models to each head when approximating the solution of the Poisson problem.

Evaluated model	Test μ	Head weights $\omega_{\ell}^{(\mu)}$			
		Head 1 ($\mu = 0.5$)	Head 2 ($\mu = 1.5$)	Head 3 ($\mu = 2.5$)	Head 4 ($\mu = 3.5$)
Two-Phase	2	0.007	0.536	0.41	0.014
One-Phase	2	0.0003	0.552	0.447	0.0001
Two-Phase	8	0.001	0.003	0.022	0.973
One-Phase	8	0.000	0.000	0.0001	0.9999

Table 4

Weights assigned by the models to each head when approximating the solution of the Heat problem and MAE for each case of study.

Evaluated model	Test μ	MAE	Head weights $\omega_{\ell}^{(\mu)}$			
			Head 1 ($\mu = 0.05$)	Head 2 ($\mu = 0.1$)	Head 3 ($\mu = 0.2$)	Head 4 ($\mu = 0.3$)
Two-Phase	0.15	0.0511	0.010	0.861	0.128	0.000
One-Phase	0.15	0.0245	0.000	0.233	0.766	0.000
Two-Phase	0.8	0.1172	0.000	0.000	0.004	0.996
One-Phase	0.8	0.1210	0.000	0.000	0.000	1.000

training parameters increases. Although these deviations remain moderate compared to more complex PDEs, they already indicate that the learned representation is primarily local and does not fully generalize beyond the observed parameter domain.

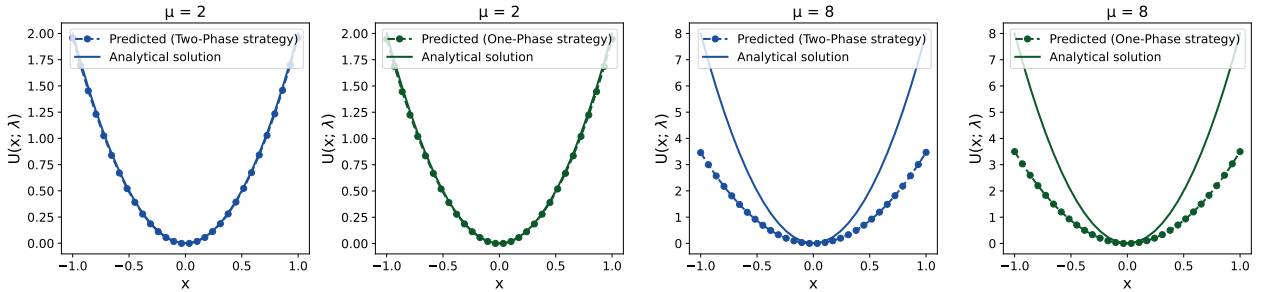


Figure 9: Evaluation of the MTL-PINN model over the Poisson parametric PDE trained with the one-phase and two-phase training strategies for values of the parameter μ close to ($\mu = 2$) and far from ($\mu = 8$) the training values $\mu \in \{0.5, 1.5, 2.5, 3.5\}$.

We next consider the Heat equation, whose results are shown in Figure 10 and Table 4. In this case, the presence of temporal dynamics introduces additional complexity in the solution manifold, although the overall behavior remains smooth. As in the Poisson case, both training strategies achieve accurate interpolation within the training range, producing solutions that align closely with the analytical evolution of the system. The one-phase strategy yields better interpolative approximations (MAE = 0.0245 vs 0.0511 achieved by the two-phase strategy), reflecting its more expressive representation of the solution space. However, extrapolation again leads to a clear degradation in performance for both training strategy. The predicted temporal evolution increasingly departs from the reference solution as the parameter moves outside the training interval, revealing limitations in the model's ability to capture the correct diffusion dynamics in unseen regimes.

Finally, we analyze the Burgers' equation, whose results are depicted in Figure 11 and Table 5. This problem presents a more challenging scenario due to its nonlinear nature and the presence of sharp gradients in the solution.

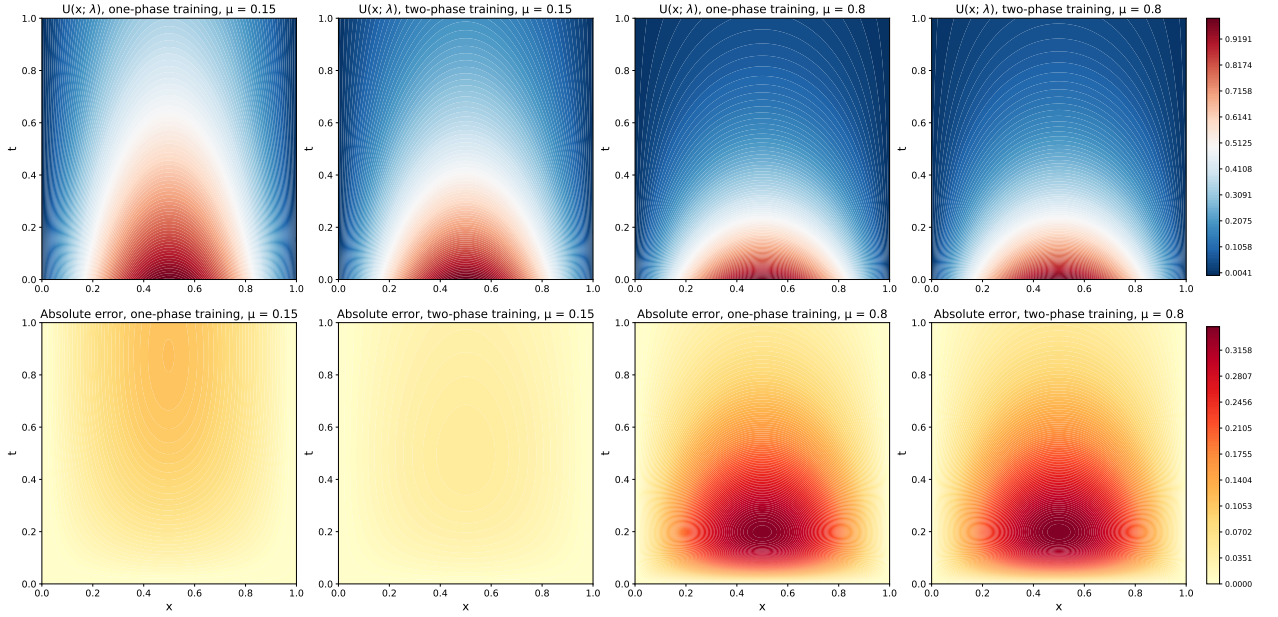


Figure 10: Evaluation of the MTL-PINN model over the Heat parametric PDE trained with the one-phase and two-phase training strategies for values of the parameter μ close to ($\mu = 0.15$) and far from ($\mu = 0.8$) the training values $\mu \in \{0.05, 0.1, 0.2, 0.3\}$ (top row: solution produced by the model; bottom row: absolute error with respect to the analytical solution).

Table 5

Weights assigned by the models to each head when approximating the solution of the Burgers problem and MAE for each case of study.

Evaluated model	Test μ	MAE	Head weights $\omega_{\ell}^{(\mu)}$			
			Head 1 ($\mu = 0.05$)	Head 2 ($\mu = 0.01$)	Head 3 ($\mu = 0.02$)	Head 4 ($\mu = 0.03$)
Two-Phase	0.015	0.0448	0.019	0.032	0.107	0.841
One-Phase	0.015	0.0349	0.246	0.165	0.196	0.392
Two-Phase	0.08	0.1970	0.000	0.001	0.037	0.962
One-Phase	0.08	0.1989	0.092	0.130	0.259	0.518

In the interpolation regime, both strategies remain reasonably accurate, although discrepancies begin to appear in regions with steep transitions. The advantage of the one-phase approach is more noticeable here, as its learned basis better captures shared patterns across parameter values. However, in extrapolation, the degradation is significantly more pronounced than in the previous PDEs. The predicted solutions fail to reproduce key qualitative features, such as the location and sharpness of gradients, indicating a loss of physical consistency. These errors are also reflected quantitatively in the higher values reported in Table 5.

Across all three PDEs, a consistent pattern emerges. Interpolation within the training parameter range is generally reliable, regardless of the specific training strategy or the complexity of the problem. In contrast, extrapolation leads to a rapid deterioration in accuracy, with the severity of the degradation increasing with the complexity and nonlinearity of the underlying PDE.

From a representation learning perspective, this behavior can be interpreted in terms of the geometry of the learned solution space. As discussed in Section 6.2, the MTL-PINN architecture effectively learns a finite set of basis functions (the heads), whose combinations approximate the solution manifold over the training parameter domain. For each PDE (Figures 9, 10 and 11), interpolation corresponds to evaluating the model within this learned manifold, where the mapping from parameters to combination weights is well supported by the training data. In contrast, extrapolation

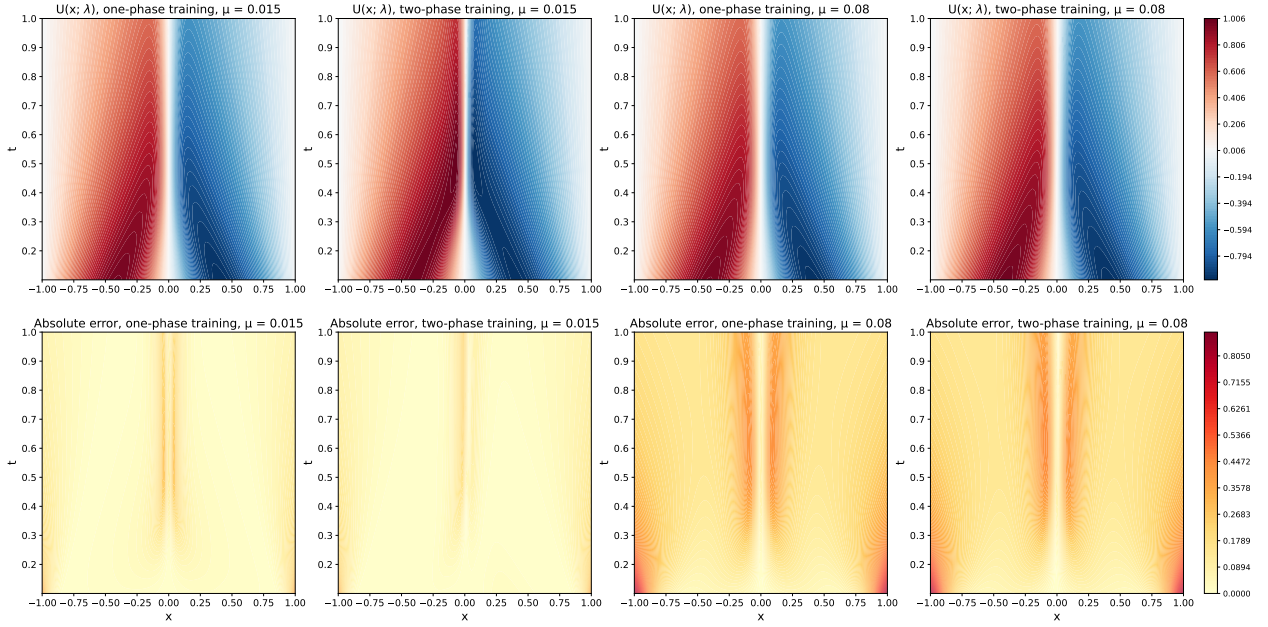


Figure 11: Evaluation of the MTL-PINN model over the Burgers' parametric PDE trained with the one-phase and two-phase training strategies for values of the parameter μ close to ($\mu = 0.015$) and far from ($\mu = 0.08$) the training values $\mu \in \{0.005, 0.01, 0.02, 0.03\}$ (top row: solution produced by the model; bottom row: absolute error with respect to the analytical solution).

requires extending this mapping beyond the observed region, where the model has not been constrained by the loss function. As a result, the voter network produces combinations that do not correspond to physically meaningful solutions.

Importantly, the comparison between training strategies reveals that the improved expressiveness of the one-phase training approach does not translate into better extrapolation behavior. While it consistently yields smoother and more accurate interpolation, particularly for more complex PDEs such as Burgers' equation (Figure 11), this advantage disappears outside the training domain. Similarly, the explicit head specialization induced by the two-phase strategy does not provide additional robustness for extrapolation. These observations suggest that the limitations observed in our experiments for RQ2 are not primarily due to architectural design or training dynamics, but rather to an intrinsic constraint of learning-based approximations under finite parameter sampling.

Taken together, the results reported here delineate the strengths and limitations of the proposed MTL-PINN architecture across different parametric PDEs. While both training strategies are able to accurately reconstruct solutions within the convex hull of the training parameters, their predictions degrade systematically when moving beyond this region, as evidenced by both the increase in approximation errors and the visible loss of fidelity in the predicted solution profiles. These findings reinforce the interpretation introduced in Section 4: the learned representations effectively span a finite-dimensional approximation of the solution manifold restricted to the training domain, but do not necessarily capture its global structure.

From a broader perspective, these results highlight a fundamental gap between representation learning and physical generalization in parametric PINNs. Although the physics-informed loss enforces consistency with the governing equations at the sampled parameter values, it does not guarantee that the learned parameter-to-solution mapping extrapolates in a physically meaningful way. Bridging this gap will require incorporating additional inductive biases that explicitly constrain the structure of the parametric dependence, such as adaptive parameter sampling strategies, regularization mechanisms on the voter network, or the inclusion of prior knowledge about the geometry of the solution manifold. Addressing these aspects remains an open research direction and is essential for extending the applicability of PINN-based approaches to scenarios where extrapolation in parameter space is unavoidable.

7. Conclusions and Future Work

In this work, we have investigated the use of a multi-task learning (MTL) architecture with multiple heads to approximate parametric families of PDE solutions. Our approach relies on learning head-specific mappings for selected parameter values, together with a voter network responsible for combining the heads to approximate solutions at new parameters. We evaluated two training strategies—Two-Phase, where heads are specialized before training the voter, and One-Phase, where both components are trained jointly—and analyzed their behavior across three representative PDE problems: Poisson, Heat, and Burgers.

RQ1 addressed whether head specialization emerges naturally in the proposed architecture. Our results show that for PDEs with smooth solutions, such as Poisson and Heat, both training strategies produce heads that specialize effectively. In contrast, for the Burgers equation, whose solutions contain sharp gradients and discontinuities, head specialization only emerges reliably under the Two-Phase strategy. When training both parts of the model simultaneously, specialization does not emerge. Instead, non-specialized heads (One-Phase model) span a subspace that captures more of the variability of the full solution family. This suggests that joint training produces a more flexible representation space, even if explicit specialization is weaker.

RQ2 investigated the interpolation/extrapolation capability of the models. For smooth PDEs, interpolation near the training space is accurate for both models. The weights assigned by the voter confirm that the models learn meaningful local interpolations: nearby training heads receive high weights while distant heads receive negligible ones. For the Burgers equation, the One-Phase model demonstrates a slight advantage when approximating parameters close to the training set, particularly around regions with discontinuities. This is likely due to the richer representational subspace revealed by the explained-variance analysis.

Across all PDEs, both training strategies exhibit limited extrapolation capability. In most cases, the voter simply selects the head corresponding to the nearest training parameter, and the model effectively behaves as a nearest-neighbor retriever rather than an interpolator. This behavior is consistent across Poisson, Heat, and Burgers, and is further confirmed by the similarity between the approximated solutions and the outputs of the closest head. Even in cases where the One-Phase model appears to distribute weights among heads, the resulting approximation typically matches that of the nearest available training parameter.

Overall, our results regarding RQ2 highlight a fundamental trade-off in the use of MTL architectures for parametric PDE learning. The two-phase strategy promotes clear head specialization, leading to interpretable models that capture parameter-specific structure. The one-phase strategy, despite lacking explicit specialization, learns a higher-dimensional representation space that can better encode solution variability for smooth and moderately complex PDEs. However, neither strategy provides robust out-of-distribution generalization in the parameter domain, and both degenerate to nearest-parameter retrieval under extrapolation.

Regarding the future work, we propose several promising research directions:

- Exploring parametric PDEs with qualitatively different behaviors. For example, the parametric Helmholtz equation, where each parameter value admits an infinite set of solutions.
- Leveraging the trained models for solution extrapolation. Approaches ranging from few-shot adaptation to more sophisticated transfer-learning strategies could help overcome the extrapolation limitations observed in our experiments.
- Designing architectures that explicitly improve parameter generalization. For example, enriching the voter network or incorporating operator-learning components capable of capturing non-local parametric relationships.

References

- Bahmani, B., Sun, W., 2021. Training multi-objective/multi-task collocation physics-informed neural network with student/teachers transfer learnings. ArXiv URL: <https://arxiv.org/abs/2107.11496>, arXiv:2107.11496.
- Baldan, M., Di Barba, P., Lowther, D.A., 2023. Physics-informed neural networks for inverse electromagnetic problems. *IEEE Transactions on Magnetics* 59, 1–5. doi:10.1109/TMAG.2023.3247023.
- Bharadwaja, B.V.S.S., Nabian, M.A., Sharma, B., Choudhry, S., Alankar, A., 2022. Physics-informed machine learning and uncertainty quantification for mechanics of heterogeneous materials. *Integrating Materials and Manufacturing Innovation* 11, 607–627. URL: <https://doi.org/10.1007/s40192-022-00283-2>, doi:10.1007/s40192-022-00283-2.
- Brenner, S.C., Scott, L.R., 2008. *The mathematical theory of finite element methods*. Springer.

- Cai, S., Mao, Z., Wang, Z., Yin, M., Karniadakis, G.E., 2021a. Physics-informed neural networks (pinns) for fluid mechanics: a review. *Acta Mechanica Sinica* 37, 1727–1738. URL: <https://doi.org/10.1007/s10409-021-01148-1>, doi:10.1007/s10409-021-01148-1.
- Cai, S., Wang, Z., Wang, S., Perdikaris, P., Karniadakis, G.E., 2021b. Physics-informed neural networks for heat transfer problems. *Journal of Heat Transfer* 143, 060801. URL: <https://doi.org/10.1115/1.4050542>, doi:10.1115/1.4050542, arXiv:https://asmedigitalcollection.asme.org/heattransfer/article-pdf/143/6/060801/6688635/ht_143_06_060801.pdf.
- Cao, W., Song, J., Zhang, W., 2024. Solving high-dimensional parametric engineering problems for inviscid flow around airfoils based on physics-informed neural networks. *Journal of Computational Physics* 516, 113285. URL: <https://www.sciencedirect.com/science/article/pii/S0021999124005333>, doi:<https://doi.org/10.1016/j.jcp.2024.113285>.
- Caruana, R., 1997. Multitask learning. *Machine Learning* 28, 41–75. URL: <https://doi.org/10.1023/A:1007379606734>, doi:10.1023/A:1007379606734.
- Cho, W., Jo, M., Lim, H., Lee, K., Lee, D., Hong, S., Park, N., 2024. Parameterized physics-informed neural networks for parameterized PDEs, in: Salakhutdinov, R., Kolter, Z., Heller, K., Weller, A., Oliver, N., Scarlett, J., Berkenkamp, F. (Eds.), *Proceedings of the 41st International Conference on Machine Learning*, PMLR. pp. 8510–8533. URL: <https://proceedings.mlr.press/v235/cho24b.html>.
- Cohen, A., DeVore, R., 2015. Approximation of high-dimensional parametric pdes. *Acta Numerica* 24, 1–159. doi:10.1017/S0962492915000033.
- Crawshaw, M., 2020. Multi-task learning with deep neural networks: A survey. URL: <https://arxiv.org/abs/2009.09796>, arXiv:2009.09796.
- Cuomo, S., Di Cola, V.S., Giampaolo, F., Rozza, G., Raissi, M., Piccialli, F., 2022. Scientific machine learning through physics-informed neural networks: Where we are and what's next. *Journal of Scientific Computing* 92, 88. URL: <https://doi.org/10.1007/s10915-022-01939-z>, doi:10.1007/s10915-022-01939-z.
- Depina, I., Jain, S., Valsson, S.M., Gotovac, H., 2022. Application of physics-informed neural networks to inverse problems in unsaturated groundwater flow. *Georisk: Assessment and Management of Risk for Engineered Systems and Geohazards* 16, 21–36. URL: <https://doi.org/10.1080/17499518.2021.1971251>, doi:10.1080/17499518.2021.1971251, arXiv:<https://doi.org/10.1080/17499518.2021.1971251>.
- Diao, Y., Yang, J., Zhang, Y., Zhang, D., Du, Y., 2023. Solving multi-material problems in solid mechanics using physics-informed neural networks based on domain decomposition technology. *Computer Methods in Applied Mechanics and Engineering* 413, 116120. URL: <https://www.sciencedirect.com/science/article/pii/S004578252300244X>, doi:<https://doi.org/10.1016/j.cma.2023.116120>.
- Difonzo, F.V., Lopez, L., Pellegrino, S.F., 2025. Physics informed neural networks for an inverse problem in peridynamic models. *Engineering with Computers* 41, 4003–4012. URL: <https://doi.org/10.1007/s00366-024-01957-5>, doi:10.1007/s00366-024-01957-5.
- Gao, H., Zahr, M.J., Wang, J.X., 2022. Physics-informed graph neural galerkin networks: A unified framework for solving pde-governed forward and inverse problems. *Computer Methods in Applied Mechanics and Engineering* 390, 114502. URL: <https://www.sciencedirect.com/science/article/pii/S0045782521007076>, doi:<https://doi.org/10.1016/j.cma.2021.114502>.
- Glas, S., Patera, A.T., Urban, K., 2020. A reduced basis method for the wave equation. *International Journal of Computational Fluid Dynamics* 34, 139–146.
- Kapoor, T., Wang, H., Núñez, A., Dollevoet, R., 2024. Physics-informed neural networks for solving forward and inverse problems in complex beam systems. *IEEE Transactions on Neural Networks and Learning Systems* 35, 5981–5995. doi:10.1109/TNNLS.2023.3310585.
- Karniadakis, G.E., Kevrekidis, I.G., Lu, L., Perdikaris, P., Wang, S., Yang, L., 2021. Physics-informed machine learning. *Nature Reviews Physics* 3, 422–440. URL: <https://doi.org/10.1038/s42254-021-00314-5>, doi:10.1038/s42254-021-00314-5.
- Kharazmi, E., Zhang, Z., Karniadakis, G.E., 2019. Variational physics-informed neural networks for solving partial differential equations. URL: <https://arxiv.org/abs/1912.00873>, arXiv:1912.00873.
- Liu, K., Luo, K., Cheng, Y., Liu, A., Li, H., Fan, J., Balachandrar, S., 2024a. Parameterized physics-informed neural networks (p-pinns) solution of uniform flow over an arbitrarily spinning spherical particle. *International Journal of Multiphase Flow* 180, 104937. URL: <https://www.sciencedirect.com/science/article/pii/S0301932224002143>, doi:<https://doi.org/10.1016/j.ijmultiphaseflow.2024.104937>.
- Liu, M., Zhang, Q., Fuentes, R., 2026. A fem-pinn approach to modelling elastoplastic soil behaviour in boundary value problems. *Computers and Geotechnics* 190, 107761. URL: <https://www.sciencedirect.com/science/article/pii/S0266352X25007104>, doi:<https://doi.org/10.1016/j.compgeo.2025.107761>.
- Liu, W., Wang, H., Xi, Z., Wang, L., Chen, C., Guo, T., Yan, M., Wang, T., 2024b. Multitask learning-driven physics-guided deep learning magnetotelluric inversion. *IEEE Transactions on Geoscience and Remote Sensing* 62, 1–16. doi:10.1109/TGRS.2024.3457893.
- Meng, Y., Chen, Y., Gao, Z., Mu, L., Sun, X., 2025. Deep learning-based reduced-order modeling for parameterized convection-dominated partial differential equations. *Physics of Fluids* 37, 027139. URL: <https://doi.org/10.1063/5.0250598>, doi:10.1063/5.0250598.
- Patera, A.T., Rozza, G., et al., 2007. Reduced basis approximation and a posteriori error estimation for parameterized partial differential equations.
- Penwarden, M., Zhe, S., Narayan, A., Kirby, R.M., 2023a. A metalearning approach for physics-informed neural networks (pinns): Application to parameterized pdes. *Journal of Computational Physics* 477, 111912. URL: <https://www.sciencedirect.com/science/article/pii/S0021999123000074>, doi:<https://doi.org/10.1016/j.jcp.2023.111912>.
- Penwarden, M., Zhe, S., Narayan, A., Kirby, R.M., 2023b. A metalearning approach for physics-informed neural networks (pinns): Application to parameterized pdes. *Journal of Computational Physics* 477, 111912. URL: <https://www.sciencedirect.com/science/article/pii/S0021999123000074>, doi:<https://doi.org/10.1016/j.jcp.2023.111912>.
- Psaros, A.F., Meng, X., Zou, Z., Guo, L., Karniadakis, G.E., 2023. Uncertainty quantification in scientific machine learning: Methods, metrics, and comparisons. *Journal of Computational Physics* 477, 111902. URL: <https://www.sciencedirect.com/science/article/pii/S0021999122009652>, doi:<https://doi.org/10.1016/j.jcp.2022.111902>.
- Quarteroni, A., Manzoni, A., Negri, F., 2015. Reduced basis methods for partial differential equations: an introduction. volume 92. Springer.
- Raissi, M., Perdikaris, P., Karniadakis, G., 2019a. Physics-informed neural networks: A deep learning framework for solving forward and inverse problems involving nonlinear partial differential equations. *Journal of Computational Physics* 378, 686–707. URL: <https://www.sciencedirect.com/science/article/pii/S0021999119300043>.

- sciencedirect.com/science/article/pii/S0021999118307125, doi:<https://doi.org/10.1016/j.jcp.2018.10.045>.
- Raissi, M., Perdikaris, P., Karniadakis, G., 2019b. Physics-informed neural networks: A deep learning framework for solving forward and inverse problems involving nonlinear partial differential equations. *Journal of Computational Physics* 378, 686–707. doi:<https://doi.org/10.1016/j.jcp.2018.10.045>.
- Rozza, G., Lassila, T., Manzoni, A., 2011. Reduced basis approximation for shape optimization in thermal flows with a parametrized polynomial geometric map, in: Hesthaven, J.S., Rønquist, E.M. (Eds.), *Spectral and High Order Methods for Partial Differential Equations*, Springer Berlin Heidelberg, Berlin, Heidelberg. pp. 307–315.
- Ruder, S., 2017. An overview of multi-task learning in deep neural networks. URL: <https://arxiv.org/abs/1706.05098>, arXiv:1706.05098.
- Sahin, T., von Danwitz, M., Popp, A., 2024. Solving forward and inverse problems of contact mechanics using physics-informed neural networks. *Advanced Modeling and Simulation in Engineering Sciences* 11, 11. URL: <https://doi.org/10.1186/s40323-024-00265-3>, doi:10.1186/s40323-024-00265-3.
- Thanasutives, P., Numao, M., Fukui, K.i., 2021. Adversarial multi-task learning enhanced physics-informed neural networks for solving partial differential equations, in: 2021 International Joint Conference on Neural Networks (IJCNN), pp. 1–9. doi:10.1109/IJCNN52387.2021.9533606.
- Wang, B., Qin, A.K., Shafiei, S., Dia, H., Mihaita, A.S., Grzybowska, H., 2023. Training physics-informed neural networks via multi-task optimization for traffic density prediction, in: 2023 International Joint Conference on Neural Networks (IJCNN), pp. 1–7. doi:10.1109/IJCNN54540.2023.10191301.
- Wu, Y., Bansal, P., Li, Y., 2025. Physics-informed multitask learning for microstructure-aware material metamodels of heterogeneous materials. *Computational Materials Science* 259, 114147. doi:<https://doi.org/10.1016/j.commatsci.2025.114147>.
- Yang, L., Meng, X., Karniadakis, G.E., 2021. B-pinns: Bayesian physics-informed neural networks for forward and inverse pde problems with noisy data. *Journal of Computational Physics* 425, 109913. URL: <https://www.sciencedirect.com/science/article/pii/S0021999120306872>, doi:<https://doi.org/10.1016/j.jcp.2020.109913>.
- Yu, J., Lu, L., Meng, X., Karniadakis, G.E., 2022. Gradient-enhanced physics-informed neural networks for forward and inverse pde problems. *Computer Methods in Applied Mechanics and Engineering* 393, 114823. URL: <https://www.sciencedirect.com/science/article/pii/S0045782522001438>, doi:<https://doi.org/10.1016/j.cma.2022.114823>.
- Zhang, X., Hu, S., 2025. A multitask dynamically interconnected learning approach for physics-informed neural networks. *Materials Today Communications* 47, 113054. URL: <https://www.sciencedirect.com/science/article/pii/S2352492825015661>, doi:<https://doi.org/10.1016/j.mtcomm.2025.113054>.
- Zhang, Y., Yang, Q., 2022. A survey on multi-task learning. *IEEE Transactions on Knowledge and Data Engineering* 34, 5586–5609. doi:10.1109/TKDE.2021.3070203.
- Zhao, Y., Yao, J., Wang, J., Xie, X., Ablameyko, S.V., 2024. Multi-task learning enhanced physics-informed neural network for solving fluid-structure interaction equations, in: 2024 IEEE 7th Information Technology, Networking, Electronic and Automation Control Conference (ITNEC), pp. 591–595. doi:10.1109/ITNEC60942.2024.10732993.